LeetCode HOT 100

PART 4

Substring

子串

10. 和为K的子数组 (No.560) MEDIUM

给你一个整数数组 `nums` 和一个整数 `k` ，请你统计并返回 *该数组中和为 `k` 的子数组的个数* 。

子数组是数组中元素的连续非空序列。

示例 1：

> **输入：** `nums = [1,1,1], k = 2`
> **输出：** `2`

示例 2：

> **输入：** `nums = [1,2,3], k = 3`
> **输出：** `2`

提示：

- $1 <= nums.length <= 2 * 10^4$
- $-1000 <= nums[i] <= 1000$
- $-10^7 <= k <= 10^7$

10. 和为K的子数组 (No.560) MEDIUM

TIPS：连续子数组之和 → 前缀和

思路

遍历前缀和，维护一个哈希表，key=前缀和+k，value=此key出现过的计数。如果某前缀和出现在哈希表，说明之前有某个前缀和+k刚好是它，那么它-前面某个前缀和就是k，从而某个子串和为k。

特别注意：(1)先更新答案再更新哈希表，否则k=0时每个前缀和会把自己更新到哈希表里，再更新答案算的其实是自己到自己的差值为0，并不是合法子串；(2)一定要小心一列数组的第一个前缀和是0，哈希表要初始化0+k计数为1

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        num_map = defaultdict(int)  # key=前缀和+k, value=数量
        num_map[k] = 1  # 前缀和初始为0
        summ, res = 0, 0
        for num in nums:
            summ += num
            if summ in num_map:  # 一定要先更新res再更新哈希表，否则k=0时出错
                res += num_map[summ]
            num_map[summ+k] += 1
        return res
```

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(n)$

11. 滑动窗口最大值 (No.239) HARD

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。

返回 *滑动窗口中的最大值* 。

示例 1：

```
输入：nums = [1,3,-1,-3,5,3,6,7], k = 3
输出：[3,3,5,5,6,7]
解释：
滑动窗口的位置                最大值
---------------               -----
[1  3  -1] -3  5  3  6  7       3
 1 [3  -1  -3] 5  3  6  7       3
 1  3 [-1  -3  5] 3  6  7       5
 1  3  -1 [-3  5  3] 6  7       5
 1  3  -1  -3 [5  3  6] 7       6
 1  3  -1  -3  5 [3  6  7]      7
```

示例 2：

```
输入：nums = [1], k = 1
输出：[1]
```

提示：

- `1 <= nums.length <= 10^5`
- `-10^4 <= nums[i] <= 10^4`
- `1 <= k <= nums.length`

11. 滑动窗口最大值 (No.239) HARD

Solution 1 – 双端单调队列

思路

维护一个双端单调队列，递减存储备选的最大值。当遍历数组中的元素时，检查一下队首元素不在窗口范围内了要出队，然后比当前值num小的所有队尾元素通通出队，它们未来比将num早出窗口，但由于num比它们大，它们永远成为不了最大值就赶紧滚，只是先前还没遇到num时它们可能是最大的。

```python
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        dq, res = deque(), []  # 双端单调队列存储备选最大值的下标
        for i in range(len(nums)):
            # 队首元素：在窗口外就出队，因为此时窗口范围是i-k+1到i
            if dq and dq[0] + k <= i:
                dq.popleft()
            # 队尾元素：比当前值小就出队，这一步保证了队列单调递减
            while dq and nums[dq[-1]] < nums[i]:
                dq.pop()
            dq.append(i)
            if i >= k-1:  # 一开始还没形成窗口直接跳过
                res.append(nums[dq[0]])
        return res
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(k)$，队首及时清理不在窗口的元素，所以队列最长为k

11. 滑动窗口最大值 (No.239) HARD

Solution 2 – 大根堆

思路

维护一个大根堆，每次取数前把堆顶不在窗口内的元素清理干净后，堆顶元素就是当前窗口的最大值。

```
class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        heap, res = [], []
        for i in range(len(nums)):
            # heapq默认小根堆，应存储负值之后取反实现大根堆
            # 存储元组时堆调整优先按第一个元素排序
            heapq.heappush(heap, (-nums[i], i))
            # 堆顶元素在窗口外要一直出队
            while heap[0][1] + k <= i:
                heapq.heappop(heap)
            if i >= k-1:  # 一开始还没形成窗口直接跳过
                res.append(-heap[0][0])
        return res
```

时间复杂度：$\mathcal{O}(n\ logn)$， n次耗时$\mathcal{O}(logn)$的堆调整

空间复杂度：$\mathcal{O}(n)$，和Solution1不一样，这里非窗口内的过期元素可能滞留在堆中

12. 最小覆盖子串 (No.76) HARD

给定两个字符串 `s` 和 `t`，长度分别是 `m` 和 `n`，返回 s 中的 **最短窗口 子串**，使得该子串包含 `t` 中的每一个字符（**包括重复字符**）。如果没有这样的子串，返回空字符串 `""`。

测试用例保证答案唯一。

示例 1：

> **输入：** s = "ADOBECODEBANC", t = "ABC"
> **输出：** "BANC"
> **解释：** 最小覆盖子串 "BANC" 包含来自字符串 t 的 'A'、'B' 和 'C'。

示例 2：

> **输入：** s = "a", t = "a"
> **输出：** "a"
> **解释：** 整个字符串 s 是最小覆盖子串。

示例 3:

> **输入：** s = "a", t = "aa"
> **输出：** ""
> **解释：** t 中两个字符 'a' 均应包含在 s 的子串中，
> 因此没有符合条件的子字符串，返回空字符串。

提示：

- `m == s.length`
- `n == t.length`
- `1 <= m, n <=` 10^5
- `s` 和 `t` 由英文字母组成

进阶： 你能设计一个在 `O(m + n)` 时间内解决此问题的算法吗？

12. 最小覆盖子串 (No.76) HARD

Solution 1 – 优化前

思路（参考@灵茶山艾府的做法）

维护一个滑动窗口和两个字符计数器t_map和window。扩展right，当window覆盖t_map说明window子串是覆盖子串，收缩left，尝试更新最小覆盖子串。

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        t_map, window = Counter(t), Counter()
        left, m = 0, len(s)
        res_left, res_right = -1, m
        for right in range(m):
            window[s[right]] += 1
            # window计数覆盖了t，说明有冗余
            while window >= t_map:
                if res_right - res_left > right - left:
                    res_left, res_right = left, right
                window[s[left]] -= 1
                left += 1
        return "" if res_left < 0 else s[res_left:res_right+1]
```

时间复杂度：$\mathcal{O}(m|\Sigma|+n)$，m为s长度，n为t长度，$|\Sigma|$为字符集大小；**空间复杂度**：$\mathcal{O}(|\Sigma|)$

12. 最小覆盖子串 (No.76) HARD

Solution 2 – 优化后

思路

（参考@灵茶山艾府的做法）

为了和[9.找到字符串中所有字母异位词]的逻辑保持基本一致，这里定义t_map是Solution1中的t_map-window，也就是window距离t_map还要消除的一个字符清单，t_map[‘a’]=2表示window还差2个a。cnt用来记录有多少个t中字符window比t_map多了，cnt=初始len(t_map)时说明window覆盖了t。注意t_map会变长，因此len(t_map)一定要提前定义一个变量l存储。t_map中所有非t中字符value是负数，因此永远不会归0影响到cnt。

```python
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        t_map = Counter(t)  # t_map是window要消除的“清单”
        left, m, l = 0, len(s), len(t_map)
        res_left, res_right = -1, m
        cnt = 0  # cnt是窗口内比t数量多的字符数
        for right in range(m):
            t_map[s[right]] -= 1
            if t_map[s[right]] == 0:
                cnt += 1
            # cnt==n 等效于Solution1中window >= t_map，即窗口覆盖了t
            while cnt == l:
                if res_right - res_left > right - left:
                    res_left, res_right = left, right
                # 收缩左边界时检查一下，刚好去掉了完美匹配的字符cnt会变
                if t_map[s[left]] == 0:
                    cnt -= 1
                t_map[s[left]] += 1
                left += 1
        return "" if res_left < 0 else s[res_left:res_right+1]
```

时间复杂度： $\mathcal{O}(m+n)$; **空间复杂度：** $\mathcal{O}(|\Sigma|)$